Solution: First Non-repeating Character (easy)

Problem Statement

Given a string, identify the position of the first character that appears only once in the string. If no such character exists, return -1.

Examples

1. **Example 1**:
   - Input: "apple"
   - Expected Output: 0
   - Justification: The first character 'a' appears only once in the string and is the first character.
2. **Example 2**:
   - Input: "abcab"
   - Expected Output: 2
   - Justification: The first character that appears only once is 'c' and its position is 2.
3. **Example 3**:
   - Input: "abab"
   - Expected Output: -1
   - Justification: There is no character in the string that appears only once.

Solution

To solve this problem, we'll use a hashmap to keep a record of each character in the string and the number of times it appears. First, iterate through the string and populate the hashmap with each character as the key and its frequency as the value. Then, go through the string again, this time checking each character against the hashmap. The first character that has a frequency of one (indicating it doesn't repeat) is your target. This character is the first non-repeating character in the string. If no such character exists, the solution should indicate that as well. This two-pass approach ensures efficiency, as each character is checked against a pre-compiled frequency map.

1. **Initialization**: Begin by creating a hashmap to store the frequency of each character in the string. This hashmap will help in identifying characters that appear only once.
2. **Frequency Count**: Traverse the string from the beginning to the end. For each character, increment its count in the hashmap.
3. **First Unique Character**: Traverse the string again from the beginning. For each character, check its frequency in the hashmap. If the frequency is 1, return its position as it's the first unique character.
4. **No Unique Character**: If the string is traversed completely without finding a unique character, return -1.

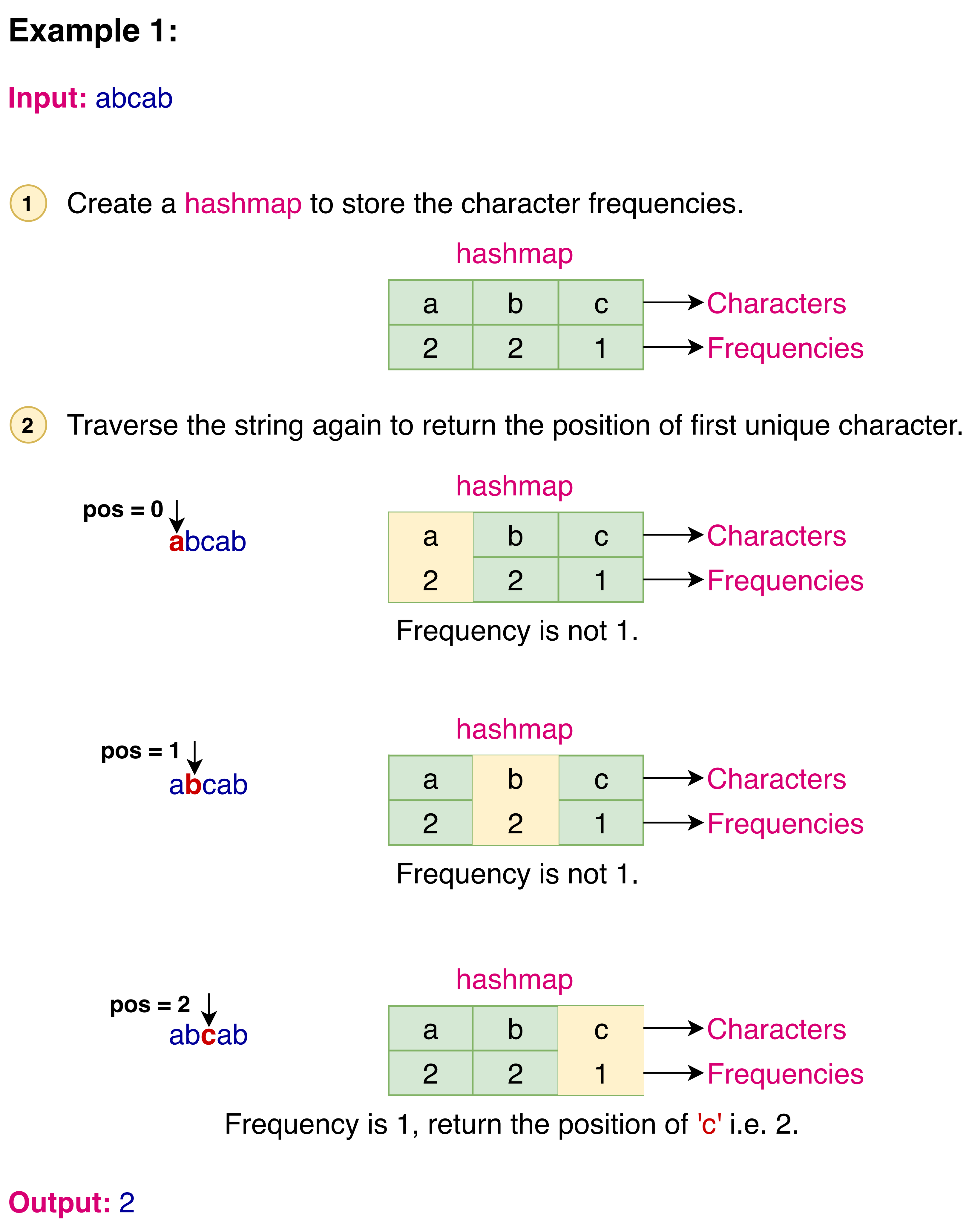
Using a hashmap ensures that we can quickly determine the frequency of each character without repeatedly scanning the string.

Algorithm Walkthrough:

Given the input string "abcab":

- Initialize a hashmap to store character frequencies.
- Traverse the string:
  - 'a' -> frequency is 1
  - 'b' -> frequency is 1
  - 'c' -> frequency is 1
  - 'a' -> frequency is 2
  - 'b' -> frequency is 2
- Traverse the string again:
  - 'a' has frequency 2
  - 'b' has frequency 2
  - 'c' has frequency 1, so return its position 2.

Here is the visual representation of the algorithm:



Example 1:

Input: abcab

① Create a hashmap to store the character frequencies.

② Traverse the string again to return the position of first unique character.

Frequency is not 1.

Frequency is not 1.

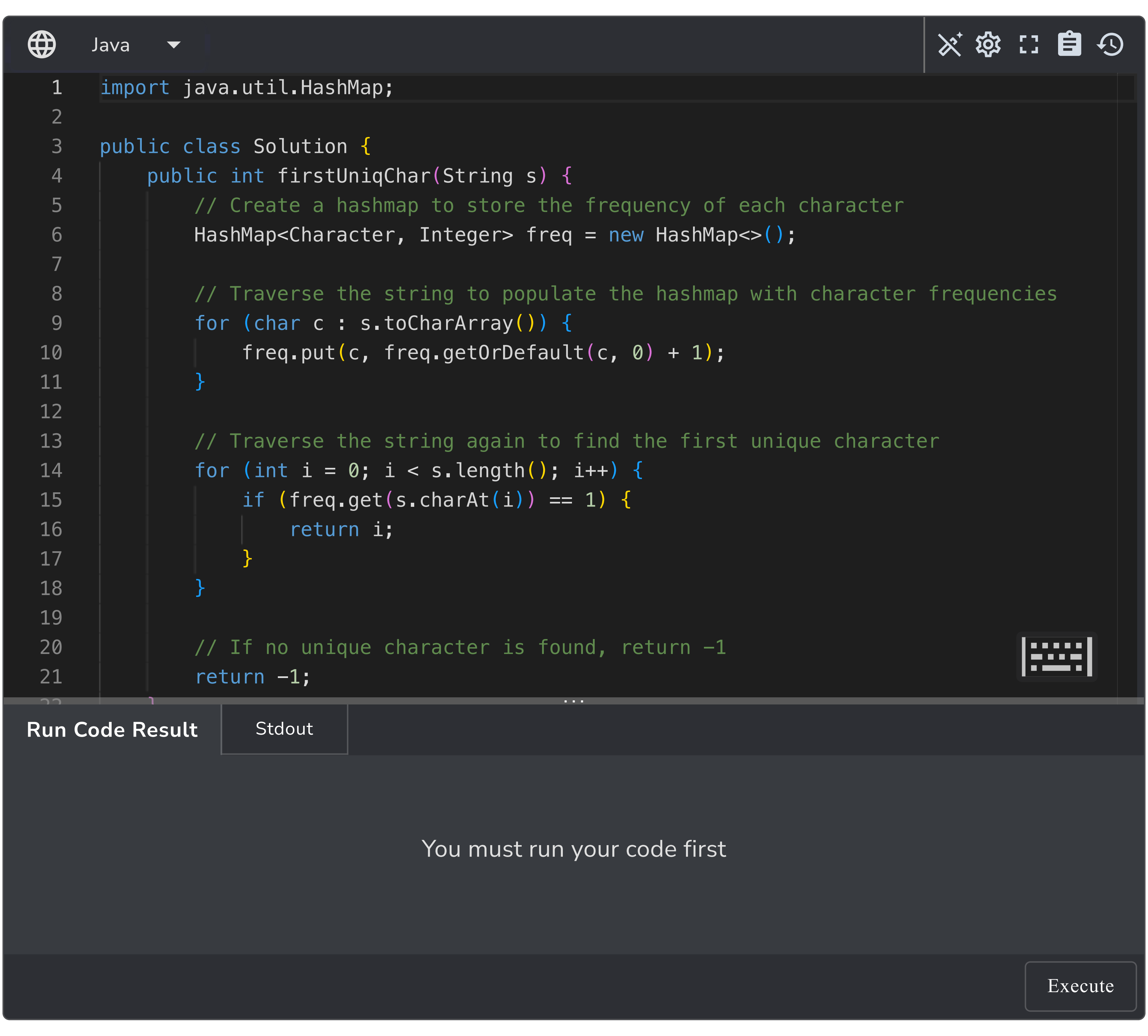
Frequency is 1, return the position of 'c' i.e. 2.

Output: 2

First Non Repeating Character

Code

Here is the code for this algorithm:



```java
import java.util.HashMap;

public class Solution {
    public int firstUniqChar(String s) {
        // Create a hashmap to store the frequency of each character
        HashMap<Character, Integer> freq = new HashMap<>();

        // Traverse the string to populate the hashmap with character frequencies
        for (char c : s.toCharArray()) {
            freq.put(c, freq.getOrDefault(c, 0) + 1);
        }

        // Traverse the string again to find the first unique character
        for (int i = 0; i < s.length(); i++) {
            if (freq.get(s.charAt(i)) == 1) {
                return i;
            }
        }

        // If no unique character is found, return -1
        return -1;
```

You must run your code first

Time Complexity

1. **Populating the hashmap with frequencies**: We traverse the entire string once to populate the hashmap with the frequency of each character. This operation takes O(n) time, where n is the length of the string.
2. **Finding the first unique character**: We traverse the string again to find the first character with a frequency of 1. This operation also takes O(n) time.

Given that these two operations are sequential and not nested, the overall time complexity is O(n) + O(n), which simplifies to O(n).

Space Complexity

1. **Hashmap for frequencies**: In the worst case, every character in the string is unique. For a string with only lowercase English letters, the maximum number of unique characters is 26. However, if we consider all possible ASCII characters, the number is 128. If we consider extended ASCII, it's 256. In any case, this is a constant number. Therefore, the space complexity for the hashmap is O(1) because it doesn't grow proportionally with the size of the input string.
2. **Input string**: The space taken by the input string is not counted towards the space complexity, as it's considered input space.

Given the above, the overall space complexity of the algorithm is O(1).

Conclusion

The approach is efficient in terms of both time and space. The time complexity is linear, which means the algorithm's runtime grows linearly with the size of the input. The space complexity is constant, indicating that the amount of additional space (memory) the algorithm uses does not grow with the size of the input.

← Previous
Problem 1: First Non-Repeating Character (Easy)

Next →
Problem 2: Largest Unique Number (Easy)

☐ Mark as Completed